

[← Go Back](#)

Hardware

Portenta H7



Tutorials

[BLE Connectivity on Portenta H7](#)[Creating GUIs with LVGL \(V7\)](#)[Dual Core Processing](#)[Creating a Flash-Optimized Key-Value Store](#)[Getting Started with OpenMV and MicroPython](#)[Debugging with Lauterbach TRACE32 GDB Front-End Debugger](#)

Arduino Portenta H7 MicroPython Cheat Sheet

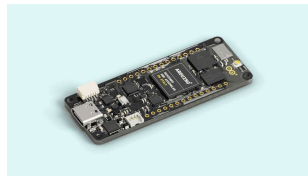
[Over-The-Air \(OTA\) Updates with the Arduino Portenta H7](#)[Reading and Writing Flash Memory](#)[Secure Boot on Portenta H7](#)[Setting Up Portenta H7 For Arduino](#)[Voice Commands With The Arduino Speech Recognition Engine](#)[Updating the Portenta Bootloader](#)[Portenta H7 as a USB Host](#)[Portenta H7 as a Wi-Fi Access Point](#)[Home](#) / [Hardware](#) / [Portenta H7](#) /

Arduino Portenta H7 MicroPython Cheat Sheet

Arduino Portenta H7 MicroPython Cheat Sheet

Learn how to set up the Arduino Portenta H7 for OpenMV. Obtain information regarding pins and how to use OpenMV and MicroPython.

Author : Benjamin Dannegård · Last revision : 01/25/2024



The Arduino Portenta H7

ON THIS PAGE

Goals

[Core](#)[Installation](#)[Pins](#)[Communication](#)[Conclusion](#)

+

+

+

Goals

This article is a collection of mini-guides and API calls that can help you get started with the Arduino Portenta H7 board. You can also visit the [documentation product page](#) for more in-depth tutorials.

Core

The Portenta H7 uses the [Arduino Mbed OS Portenta Boards core](#).

Installation

[Help](#)

Arduino supports the OpenMV build of MicroPython to be used with the Portenta H7. To install it on your board, you can check out the guide below:

[Installing the Arduino Portenta H7 with MicroPython](#)

Potential Issues

Sometimes the board is not detected even when the board is connected to your computer. This can be solved through the following steps:

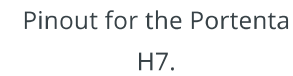
- 1 Disconnect the board from your computer by removing the USB cable.
- 2 Reconnect the board to your computer.
- 3 If it still doesn't show up, double-press the reset button, to activate the bootloader mode.
- 4 Re-install the OpenMV firmware

Libraries

Some really useful MicroPython libraries in OpenMV are `pyb` and `machine`. Within these two libraries, there are plenty of modules that will help make MicroPython scripting easier. These libraries do not need to be downloaded, they are installed with OpenMV. They only need to be included in your script.

Pins

The below graphic shows the MKR



Pin/ Function	Port
PA0	PA0
PA1	PA1
PA2	PA2
PA3	PA3
PA4	PA4
PA5	PA5
PA6	PA6
PA7	PA7
PA8	PA8
PA9	PA9
PA10	PA10
PA11	PA11
PA12	PA12
PA13	PA13

Pin/ Function	Port
PA15	PA15
PB0	PB0
PB1	PB1
PB2	PB2
PB3	PB3
PB4	PB4
PB5	PB5
PB6	PB6
PB7	PB7
PB8	PB8
PB9	PB9
PB10	PB10
PB11	PB11
PB12	PB12
PB13	PB13
PB14	PB14
PB15	PB15
PC0	PC0
PC1	PC1
PC2	PC2
PC3	PC3
PC4	PC4
PC5	PC5
PC6	PC6
PC7	PC7
PC8	PC8
PC9	PC9
PC10	PC10
PC11	PC11
PC12	PC12
PC13	PC13
PC14	PC14
PC15	PC15

Pin/ Function	Port
PD1	PD1
PD2	PD2
PD3	PD3
PD4	PD4
PD5	PD5
PD6	PD6
PD7	PD7
PD8	PD8
PD9	PD9
PD10	PD10
PD11	PD11
PD12	PD12
PD13	PD13
PD14	PD14
PD15	PD15
PE0	PE0
PE1	PE1
PE2	PE2
PE3	PE3
PE4	PE4
PE5	PE5
PE6	PE6
PE7	PE7
PE8	PE8
PE9	PE9
PE10	PE10
PE11	PE11
PE12	PE12
PE13	PE13
PE14	PE14
PE15	PE15
PF0	PF0
PF1	PF1

Pin/ Function	Port
PF3	PF3
PF4	PF4
PF5	PF5
PF6	PF6
PF7	PF7
PF8	PF8
PF9	PF9
PF10	PF10
PF11	PF11
PF12	PF12
PF13	PF13
PF14	PF14
PF15	PF15
PG0	PG0
PG1	PG1
PG2	PG2
PG3	PG3
PG4	PG4
PG5	PG5
PG6	PG6
PG7	PG7
PG8	PG8
PG9	PG9
PG10	PG10
PG11	PG11
PG12	PG12
PG13	PG13
PG14	PG14
PG15	PG15
PH0	PH0
PH1	PH1
PH2	PH2
PH3	PH3

Pin/ Function	Port
PH5	PH5
PH6	PH6
PH7	PH7
PH8	PH8
PH9	PH9
PH10	PH10
PH11	PH11
PH12	PH12
PH13	PH13
PH14	PH14
PH15	PH15
PI0	PI0
PI1	PI1
PI2	PI2
PI3	PI3
PI4	PI4
PI5	PI5
PI6	PI6
PI7	PI7
PI8	PI8
PI9	PI9
PI10	PI10
PI11	PI11
PI12	PI12
PI13	PI13
PI14	PI14
PI15	PI15
PJ0	PJ0
PJ1	PJ1
PJ2	PJ2
PJ3	PJ3
PJ4	PJ4
PI5	PI5

Pin/ Function	Port
PJ7	PJ7
PJ8	PJ8
PJ9	PJ9
PJ10	PJ10
PJ11	PJ11
PJ12	PJ12
PJ13	PJ13
PJ14	PJ14
PJ15	PJ15
PK0	PK0
PK1	PK1
PK2	PK2
PK3	PK3
PK4	PK4
PK5	PK5
PK6	PK6
PK7	PK7
UART1_TX	PA9
UART1_RX	PA10
UART4_TX	PA0
UART4_RX	PI9
UART6_TX	PG14
UART6_RX	PG9
UART8_TX	PJ8
UART8_RX	PJ9
ETH_RMII_REF_CLK	PA1
ETH_MDIO	PA2
ETH_RMII_CRS_DV	PA7
ETH_MDC	PC1
ETH_RMII_RXD0	PC4
ETH_RMII_RXD1	PC5
ETH_RMII_TX_EN	PG11
ETH_RMII_TXD0	PG13

Pin/ Function	Port
USB_HS_CLK	PA5
USB_HS_STP	PC0
USB_HS_NXT	PH4
USB_HS_DIR	PI11
USB_HS_D0	PA3
USB_HS_D1	PB0
USB_HS_D2	PB1
USB_HS_D3	PB10
USB_HS_D4	PB11
USB_HS_D5	PB12
USB_HS_D6	PB13
USB_HS_D7	PB5
USB_HS_RST	PJ4
USB_DM	PA11
USB_DP	PA12
BOOT0	BOOT0
DAC1	PA4
DAC2	PA5
LEDR	PK5
LEDG	PK6
LEDB	PK7
I2C1_SDA	PB7
I2C1_SCL	PB6
I2C3_SDA	PH8
I2C3_SCL	PH7
-WL_REG_ON	PJ1
-WL_HOST_WAKE	PJ5
-WL_SDIO_0	PC8
-WL_SDIO_1	PC9
-WL_SDIO_2	PC10
-WL_SDIO_3	PC11
-WL_SDIO_CMD	PD2
-WL_SDIO_CLK	PC12

Pin/ Function	Port
-BT_TXD	PA15
-BT_CTS	PF9
-BT_RTS	PF8
-BT_REG_ON	PJ12
-BT_HOST_WAKE	PJ13
-BT_DEV_WAKE	PJ14
-QSPI2_CS	PG6
-QSPI2_CLK	PF10
-QSPI2_D0	PD11
-QSPI2_D1	PD12
-QSPI2_D2	PF7
-QSPI2_D3	PD13

I/O Pins

To access the I/O pins, you can use the `Pin` module from the `pyb` library.

```
1 from pyb import Pin
```

To reference a pin on the Portenta, you can use the `Pin()` constructor. The first argument you have to provide is the pin you want to use. The second parameter, `mode`, can be set as: `Pin.IN`, `Pin.OUT_PP`, `Pin.OUT_OD`, `Pin.AF_PP`, `Pin.AF_OD` or `Pin.ANALOG`. An explanation of the pin modes can be found [here](#). The third parameter, `pull`, represents the pull mode. It can be set to: `Pin.PULL_NONE`, `Pin.PULL_UP` or `Pin.PULL_DOWN`. E.g.:

```
1 pin0 = Pin('P0', mode, pu
```

To get the logic level of a pin, call `.value()`. It will return a 0 or a 1. This corresponds to `LOW` and `HIGH` in Arduino terminology.

```
1 pin0.value()
```

PWM

To use PWM, you import the `pyb`, `time`, `Pin`, `Timer` modules.

```
1 import pyb
2 import time
3 from pyb import Pin, Time
```

First you need to choose the pin you want to use PWM with.

```
1 pin1 = Pin("PC6", Pin.OUT
```

Create a timer for the PWM, where you set the ID and the frequency.

```
1 timer1 = Timer(3, freq=10
```

Then you need to start a PWM channel with the timer object.

```
1 channel1 = timer1.channel
```

Get or set the pulse width value on a channel. To get, pass no arguments. To set, give a value as an argument.

```
1 channel1.pulse_width(Widt
```

RGB LED

The Portenta H7 has built-in RGB that can be used as feedback for applications. Using the `pyb` library, you can easily define the different LED colors on the Portenta.

For this you will use the `pyb` library.

```
1 import pyb
```

Now you can easily define the different colors of the built in LED.

```
1 redLED = pyb.LED(1)
2 greenLED = pyb.LED(2)
3 blueLED = pyb.LED(3)
```

And then control them in our script.

```
1 redLED.on()
2 redLED.off()
3
4 greenLED.on()
5 greenLED.off()
6
7 blueLED.on()
8 blueLED.off()
```

You could also set a custom intensity for our LED lights. This ranges between the values 0 (off) and 255 (full on). Below you can see an example of how to set the intensity on our different LED lights.

```
1 redLED.intensity(128)
2 greenLED.intensity(64)
3 blueLED.intensity(50)
```

If no argument is given in the `.intensity()` function, it will return the LED intensity.

Communication

Like other Arduino® products, the Portenta H7 features dedicated pins for different protocols.

SPI

The pins used for SPI on the Portenta H7 are the following:

Pin	Function
PI0	CS
PC3	COPI
PI1	CK
PC2	CIPO

You can refer to the [pinout](#) above to find them on the board.

First, you have to import the relevant module from `pyb`.

```
1 from pyb import SPI
```

When you initialize SPI, the only thing you need to state is the bus, which will always be `2` on the Portenta H7; this is the only available bus. The rest of the arguments are optional. But if it is needed, you can state the mode of the SPI device as either `SPI.MASTER` or `SPI.SLAVE`, you can also manually set the `baudrate` of the device. `Polarity` can be set to 0 or 1, and is the logic level the idle clock line sits at (HIGH or LOW). `Phase` can be 0 or 1 to sample data on the first (0) or second (1) clock edge.

```
1 spi = SPI(2, SPI.MASTER,
```

Now, if you want to send data over SPI, you simply call `.send()` inside the arguments you want to send. `data` is the data to send, which could be an integer (`dataInt`) or a buffer object (`dataBuffer`). It is optional to set the `timeout`, it indicates the timeout in milliseconds to wait for the send.

```
1 dataInt = 21
2 dataBuffer = bytearray(4)
```

Similarly, if you want to receive data over SPI, you call `.recv()`. `data` indicates the number of bytes to receive, this can be an integer (`dataInt`) or a buffer (`dataBuffer`), which will be filled with received bytes. It is optional to set the `timeout`, which is the time in milliseconds to wait for the receive.

```
1 dataInt = 0
2 dataBuffer = bytearray(4)
3 SPI.recv(data, timeout=50)
```

I2C

The pins used for I2C (Inter-Integrated Circuit) on the Portenta H7 are the following:

Pin	Function
PH8	SDA
PH7	SCL

You can refer to the [pinout](#) above to find them on the board.

To use the I2C, you import the relevant module.

```
1 from pyb import I2C
```

You can now create the I2C object. To create an I2C object you need to state the bus, this indicates what pins you will use for I2C. Giving bus a value of `3` starts I2C on the SCL and SDA pins on the Portenta H7.

```
1 i2c = I2C(3)
```

Now that the object is created, you can initialize it. You need to decide if your device is going to be a controller (I2C.MASTER) or a reader (I2C.SLAVE). If it is a reader device, you also need to set the `address`. You can then set a baudrate if you need to.

```
1 i2c.init(I2C.MASTER, addr
```

To receive data on the bus, you call the `.recv()` function. In the functions arguments `data` is the number of bytes to receive, it can be an integer (`dataInt`) or a buffer (`dataBuffer`), which will be filled with received bytes. `addr` is the address to receive from, this is only required in controller mode. `timeout` indicates how many milliseconds to wait for the receive. The code below shows how to receive and print your data in the OpenMV serial terminal.

```
1 dataInt = 0
2 dataBuffer = bytearray(4)
3 receivedData = i2c.recv(d
4 Print(receivedData)
```

To send data on the bus, you can call the `.send()` function. In the functions arguments `data` is the data to send, an integer (`dataInt`) or

required in controller mode.

`timeout` indicates how many milliseconds to wait for the send.

```
1 dataInt = 412
2 dataBuffer = bytearray(4)
3 i2c.send(data, addr=0, ti
```

If you need to make sure that devices are connected to the I2C bus, you can use the `.scan()` function. It will scan all I2C addresses from 0x01 to 0x7f and return a list of those that respond. It only works when in controller mode.

```
1 i2c.scan()
```

UART

The pins used for UART on the Portenta H7 are the following:

Pin	Function
PA10	RX
PA9	TX

You can refer to the [pinout](#) above to find them on the board.

To use the UART, you need to import the relevant module.

```
1 from pyb import UART
```

To create the UART object, you need

there is only one UART bus available to use with OpenMV through the board's pins.

```
1 uart = UART(1)
```

With the object created, you can initialize it with `init`. When initializing, you can set the `baudrate`. `bits` is the number of bits per character (7, 8 or 9). `parity` can be set to `None`, `0` (even) or `1` (odd). `stop` is the number of stop bits, 1 or 2. `flow` sets the flow control type, can be 0, `UART.RTS`, `UART.CTS` or `UART.RTS | UART.CTS`. More information on this can be found [here](#). `timeout` is the time in milliseconds to wait for writing/reading the first character. `timeout_char` is the timeout in milliseconds to wait between characters while writing or reading. `read_buf_len` is the character length of the read buffer (0 to disable).

```
1 uart.init(baudrate, bits=
```

To read from UART, you can call `.read()`. If `bytes` is specified then read at most that many bytes. If `bytes` is not given then the method reads as much data as possible. It returns after the timeout has elapsed. The example code below will read bytes received through `uart` into an array and then print it in the serial terminal.

```
1 array = bytearray(5)
2 uart.read(array)
3 print(array)
```

If you intend to write over UART, you can call `.write()`. The function writes `buffer` of bytes to the bus. If characters are 7 or 8 bits wide then each byte is one character. If characters are 9 bits wide then two bytes are used for each character and `buffer` must contain an even number of bytes.

```
1 uart.write(buffer)
```

WIFI

To use Wi-Fi you first need to import the relevant library.

```
1 import network
```

Then you need to define the Wi-Fi networks SSID and put that in a variable. You must do the same for the networks password.

```
1 SSID=' '
2 PASSWORD=' '
```

Next, you can create a WLAN network interface object. In the argument you can enter `network.STA_IF`, which indicates

```
1 wlan = network.WLAN(netwo
```

To activate the network interface, you can simply call `.activate` with the argument `True`.

```
1 wlan.active(True)
```

Now you can decide which network to connect to. Here it is where the `SSID` and `PASSWORD` variables come in handy.

```
1 wlan.connect(SSID, PASSWO
```

If you need to troubleshoot, the connection `.status()` can be used. This function will return a value that describes the connection status. It will also let you know what went wrong with the connection in case it failed.

```
1 wlan.status()
```

Audio

If you want to use audio with the Portenta H7, you first need to include the `audio` module. Another helpful module is `micro_speech`, this runs Google's TensorFlow Lite

```
1 import audio, micro_speech
```

Next you need to initialize the audio object. In the initialization you can decide how many `channels` to use, it is possible to use either 1 or 2 channels. Frequency decides the sample frequency. Using a higher sample frequency results in a higher noise floor, meaning less effective bits per sample. By default audio samples are 8-bits with 7-bits of effective dynamic range. `gain_db` sets the microphone gain to use. `highpass` is the high pass filter cut off given the target sample frequency.

```
1 audio.init(channels=2, fr
```

If you need to deinitialize the audio object, you can simply call `deint()`.

```
1 audio.deint()
```

To use `micro_speech`, you first need to create a `micro_speech` object. You can create this object in the variable `speech`.

```
1 speech = micro_speech.Mic
```

Next you can start streaming audio

this you can call

`audio.start_streaming()`. Here you can pass the `micro_speech` object as the argument, this will fill the object with audio samples. The MicroSpeech module will compute the FFT of the audio samples and keep a sliding window internally of the FFT the last 100ms or so of audio samples received as features for voice recognition.

```
1 audio.start_streaming(spe
```

If you need to stop the audio streaming, you can call

`.stop_streaming()`.

```
1 audio.stop_streaming()
```

Conclusion

This cheat sheet is written as a quick reference mainly to look up the features of this product. For a more in-depth walk through experience please have a look at the other tutorials.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0

If you see
anything
wrong, you
can edit
this page
[here](#).

Was this article helpful?

